AIR QUALITY DATA ANALYSIS

A Python-Based Data Analysis Project

Submitted by: KA Sunaina

Course: AI-Driven Data Analytics

1. ABSTRACT

This project, titled **Air Quality Data Analysis** focuses on analysing air quality monitoring data collected from different locations. The dataset is cleaned by handling missing values, duplicates, inconsistent values, and data type issues. Invalid geographical coordinates and potential outliers in pollutant measurements are also examined. Exploratory Data Analysis (EDA) is performed using summary statistics and pollutant-wise and location-wise analysis. Data visualization techniques are used to compare pollutant measurements, identify patterns, and explore the geographical distribution of monitoring stations. The main purpose of this project is to transform raw air quality data into meaningful information through data cleaning, statistical analysis, and visualization. The analysis helps identify patterns and variations in the available pollutant readings across different locations.

2. INTRODUCTION

The **Air Quality Data Analysis** project focuses on exploring and analysing air quality monitoring data using data analytics techniques. The dataset contains information about countries, states, cities, monitoring stations, geographical coordinates, pollutant types, and minimum, maximum, and average pollutant measurements.

The project begins with data loading and cleaning, including handling missing values, removing duplicate records where necessary, correcting inconsistent values, and converting columns to appropriate data types. Potential outliers in pollutant measurements are examined using the Interquartile Range (IQR) method. Additional time-based columns are created to support further analysis.

Exploratory Data Analysis (EDA) is then performed using summary statistics and pollutant-wise and location-wise comparisons. Charts and graphs created with Matplotlib, Seaborn, and Plotly help present the results and explore patterns in pollutant measurements and the geographical distribution of monitoring stations.

The main purpose of this project is to apply data analytics techniques to clean, analyse, and visualize air quality data. It aims to identify useful patterns and variations in pollutant measurements across different locations and present the findings through charts, graphs, and statistical summaries.

3. Problem Statement

Air pollution is a major environmental concern that affects human health and the environment. The dataset contains air quality readings collected from monitoring stations across different locations in India. Analysing this data helps identify pollutant patterns, compare air quality readings across states, and understand relationships between numerical variables. Therefore, this project aims to analyse the dataset using Python and visualisation techniques to obtain meaningful insights.

4. PROJECT OBJECTIVES

The main objectives of this project are:

- To clean the air quality dataset by handling missing values, duplicate records, and inconsistent data.
- To convert data columns into appropriate data types for analysis.
- To identify potential outliers in pollutant measurements using the IQR method.
- To perform Exploratory Data Analysis (EDA) using summary statistics.
- To analyse pollutant measurements across different pollutant types and locations.
- To explore the geographical distribution of air quality monitoring stations using latitude and longitude.
- To analyse relationships between numerical variables using correlation analysis.
- To create charts and graphs to visualise the results.
- To identify meaningful patterns and variations in the available air quality data.

5. DATASET DESCRIPTION

The dataset used in this project was obtained from Data.gov.in. It contains air quality monitoring information collected from different locations in India. The dataset includes geographical details of monitoring stations, pollutant types, and pollutant measurements.

Before data cleaning, the dataset contained 3,563 records. After cleaning and preprocessing, the dataset contained 3,556 records. The exact number of columns was identified using the shape attribute in Pandas.
Important Columns:

- Country: The country where the monitoring station is located.
- State: The state in which the monitoring station is situated.
- City: The city where the monitoring station is located.
- Station: The name of the air quality monitoring station.
- Last Update: The date and time of the recorded measurement.
- Latitude: The geographical latitude of the monitoring station.
- Longitude: The geographical longitude of the monitoring station.
- Pollutant ID: Identifies the pollutant being measured, such as PM2.5, PM10, and NH3.
- Pollutant Min: The minimum recorded pollutant value.
- Pollutant Max: The maximum recorded pollutant value.
- Pollutant Avg: The average recorded pollutant value.

These columns were used for data cleaning, exploratory data analysis, statistical analysis, and visualisation of air quality patterns across different locations.

6. TOOLS AND TECHNOLOGIES USED

The following tools and libraries are used in this project:

6.1 Google Colab

Google Colab is used to write and execute Python code for data cleaning, analysis, and visualizations.

6.2 Python

Python is the programming language used to process and analyse the air quality dataset.

6.3 Pandas

Pandas is used for data loading, cleaning, handling missing values and duplicates, converting data types, and performing data analysis

6.4 NumPy

NumPy is used for numerical operations and calculations.

6.5 Matplotlib

Matplotlib is used to create charts and graphs to present the results of the analysis.

6.6 Seaborn

Seaborn is used to create statistical visualizations, such as distribution plots, box plots, and correlation heatmaps.

6.7 Plotly

Plotly is used to create interactive visualizations, including geographical scatter plots that display monitoring stations using latitude and longitude.

7. DATA LOADING AND INITIAL INSPECTION

The air quality dataset was loaded into Python using the Pandas library. Initial inspection was performed to understand the dataset structure, identify the columns, check data types, and examine summary statistics before cleaning.

The following Pandas methods were used:

- **Head ()** – Displays the first five rows of the dataset.
- The **`tail()`** method displays the last five rows
- **`columns`** and **`dtypes`** show the column names and data types.
- The **`isnull().sum()`** method checks for missing values
- **`Duplicated().sum()`** counts duplicate records.
- **Info ()** – Shows column names, data types, and non-null counts.
- **shape** – Returns the number of rows and columns in the dataset.
- **Describe ()** – Provides summary statistics for numerical columns, including the mean, minimum, maximum, and standard deviation.

  Before cleaning, the dataset contained 3,563 records. These inspection steps helped understand the dataset and identify areas requiring further checking and preprocessing.

8. DATA CLEANING AND PREPROCESSING

Data cleaning was performed to improve the accuracy and consistency of the

dataset before analysis.

8.1 Handling Missing Values

Missing values were checked to understand whether any columns contained incomplete data. The results were reviewed before deciding whether any treatment was necessary.

8.2 Removing Duplicate Records

Duplicate records were checked to identify repeated entries. Any duplicate records were removed only if they were confirmed during inspection.

8.3 Correcting Data Types

The data types of the columns were inspected using Pandas. Numerical columns were reviewed to ensure they could be used for statistical calculations, while date and categorical columns were checked for appropriate formats.

8.4 Identifying Outliers Using the IQR Method

The Interquartile Range (IQR) method can be used to identify potential outliers in numerical pollutant measurements. Values outside the calculated lower and upper bounds were examined. Potential outliers should be reviewed before deciding whether to retain or remove them.

8.5 Checking Invalid Values

The latitude and longitude columns were checked against their valid ranges:

- **Latitude:** −90 to 90 degrees.
- **Longitude:** −180 to 180 degrees.

Incorrect coordinate values were reviewed and corrected using verified coordinates for the corresponding monitoring station.

8.6 Correcting Inconsistencies in Categorical Columns

The country column contained both **India** and **INDIA**. The inconsistent value **INDIA** was replaced with **India**.

The state column was checked for unnecessary leading and trailing spaces. Seven values contained extra spaces, which were removed using the strip() function. The city column was also cleaned by removing unnecessary leading and trailing spaces.

8.7 Handling Incorrect or Irrelevant Data

Incorrect coordinate values were reviewed and corrected where the appropriate station coordinates could be verified. The dataset was then prepared for further analysis and visualisation.

After cleaning and preprocessing, the dataset contained **3,556 records**.

9. EXPLORATORY DATA ANALYSIS (EDA)

Exploratory Data Analysis (EDA) is performed to understand the structure, characteristics, and patterns in the air quality dataset. It helps identify variations in pollutant measurements and prepare the data for further analysis.

The following analyses are carried out in this project:

9.1 Summary Statistics

- Used the describe () function to examine the count, mean, standard deviation, minimum, maximum, and quartiles of numerical columns.

9.2 Pollutant-Wise Analysis

- Examined the number of records and average recorded values for different pollutant types.

9.3 Location-Wise Analysis

- Compared records across states and cities and examined the distribution of monitoring stations.

9.4 Categorical Data Analysis

- Used value_counts () and mode () to examine categorical columns such as country, state, city, station, and pollutant ID.

9.5 Distribution Analysis

- Used histograms and box plots to understand the distribution and variation of pollutant readings and identify potential outliers.

9.6 Percentile Analysis

- The 25th, 50th (median), and 75th percentiles were examined using descriptive statistics to understand the spread and distribution of pollutant measurements.

9.7 Percentage Analysis

- Percentage analysis was performed using value_counts (normalize=True) to understand the distribution of records across categorical columns such as country, state, city, station, and pollutant type. India accounted for 100% of records, while Maharashtra had the highest state share (16.93%) and Delhi had the highest city share (8.63%).

9.8 Correlation Analysis

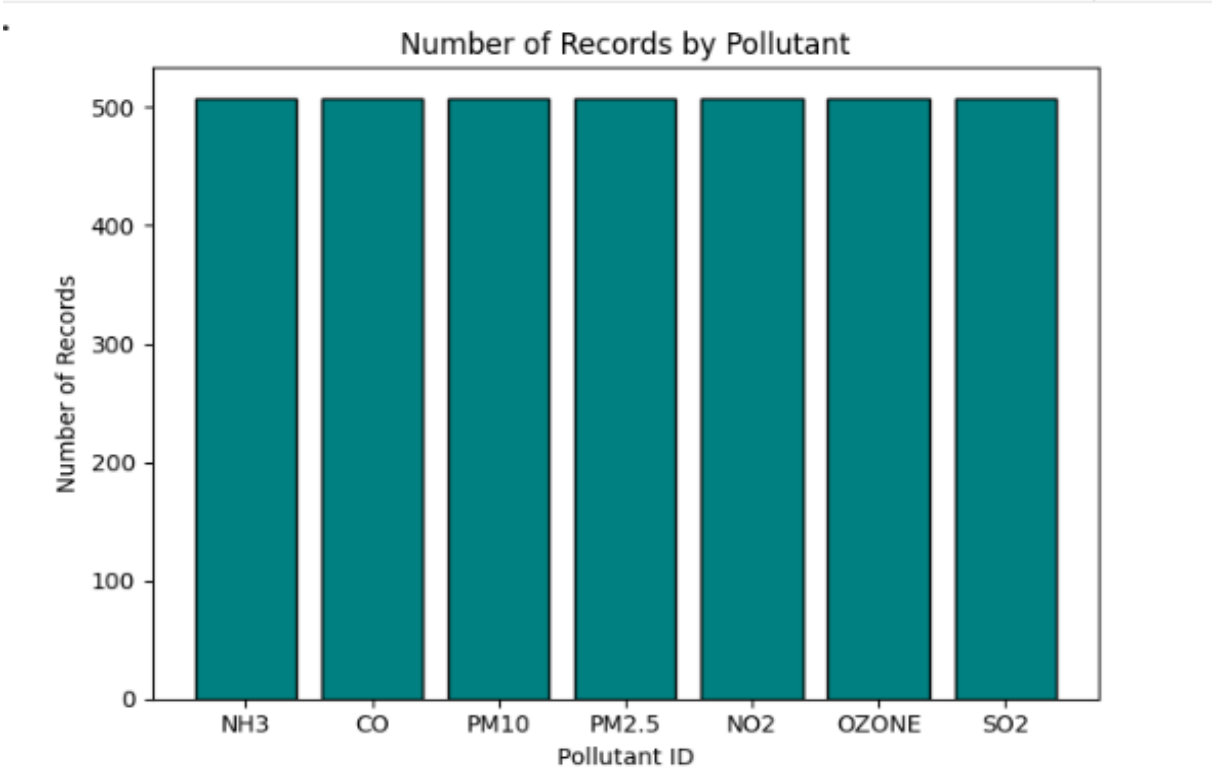
- Examined relationships between numerical variables using a correlation matrix. Strong positive correlations were observed between minimum and average pollutant values and between maximum and average pollutant values.

10. DATA VISUALIZATION AND INTERPRETATION

1. Bar Chart: Number of Records by Pollutant

Library: Matplotlib | **Analysis:** Univariate (frequency of one categorical variable)

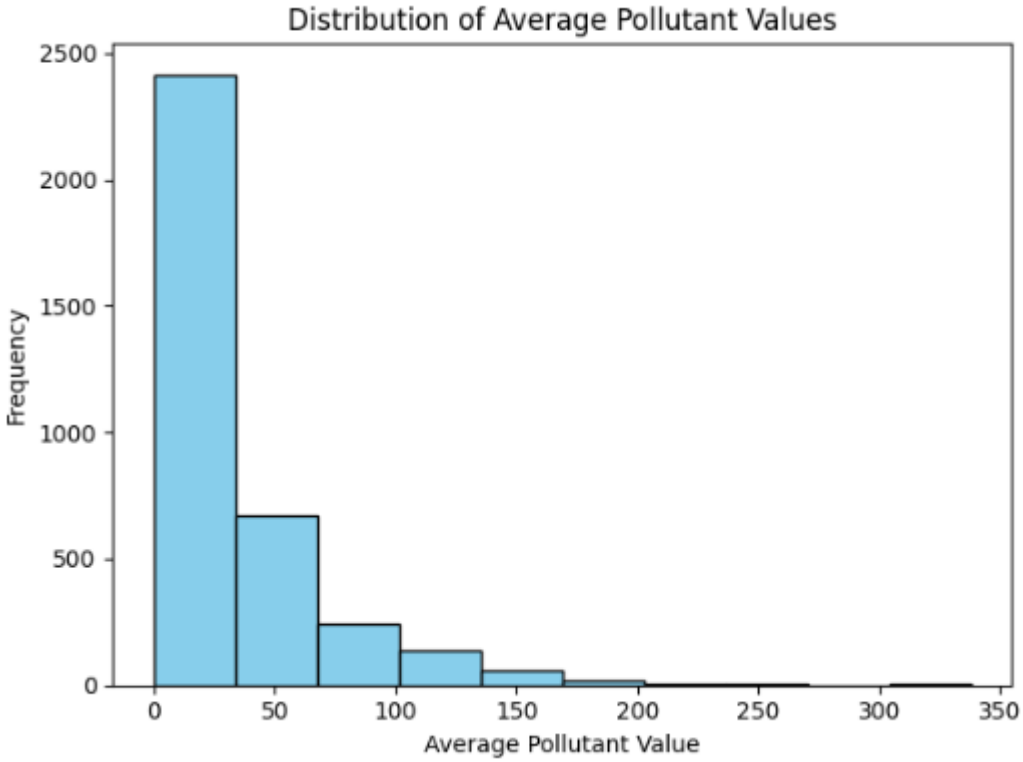
This bar chart shows the number of records for each pollutant type, including NH3, CO, PM10, PM2.5, NO2, OZONE, and SO2. The record counts are similar across pollutant categories, indicating that the dataset contains a comparable number records for each type.



2. Distribution of Average Pollutant Values

Library: Matplotlib | **Analysis:** Univariate

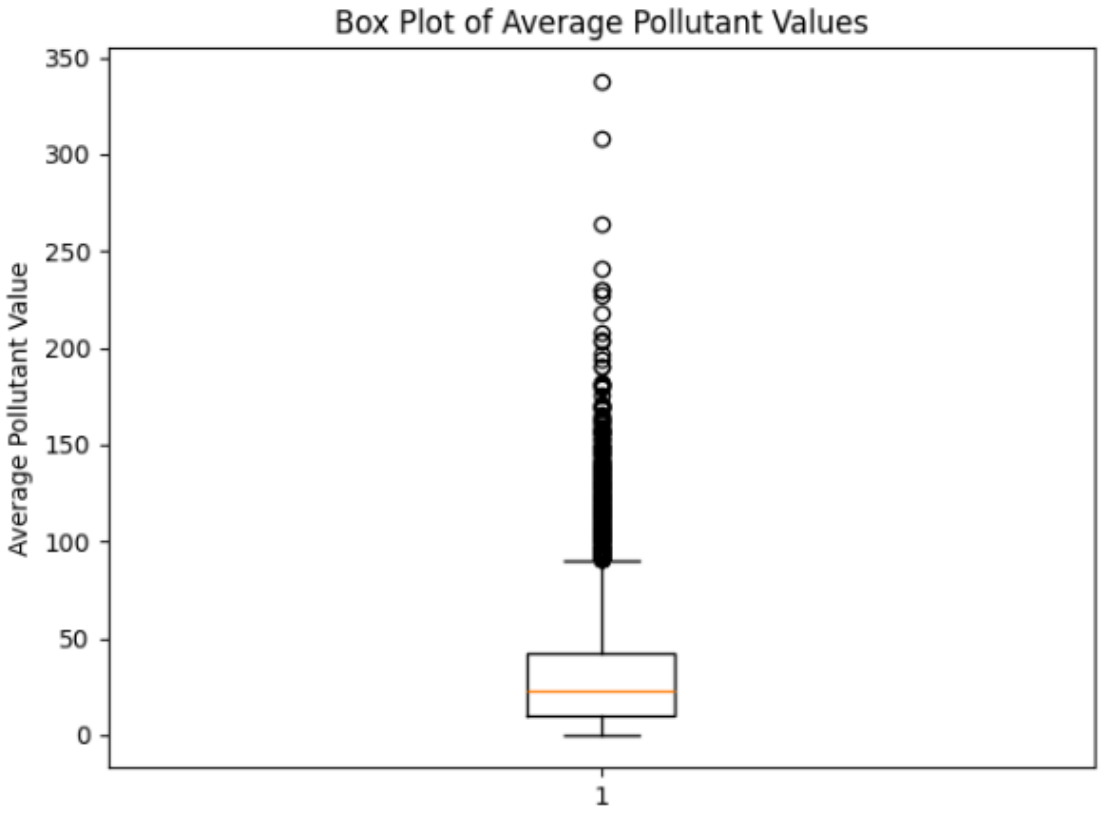
This histogram shows the distribution of average pollutant readings. Most readings are concentrated at lower values, while fewer observations occur at higher value. The distribution is right-skewed, indicating the presence of some high pollutant Readings.



3. Box Plot: Distribution of Average Pollutant Values

Library: Matplotlib | **Analysis:** Univariate

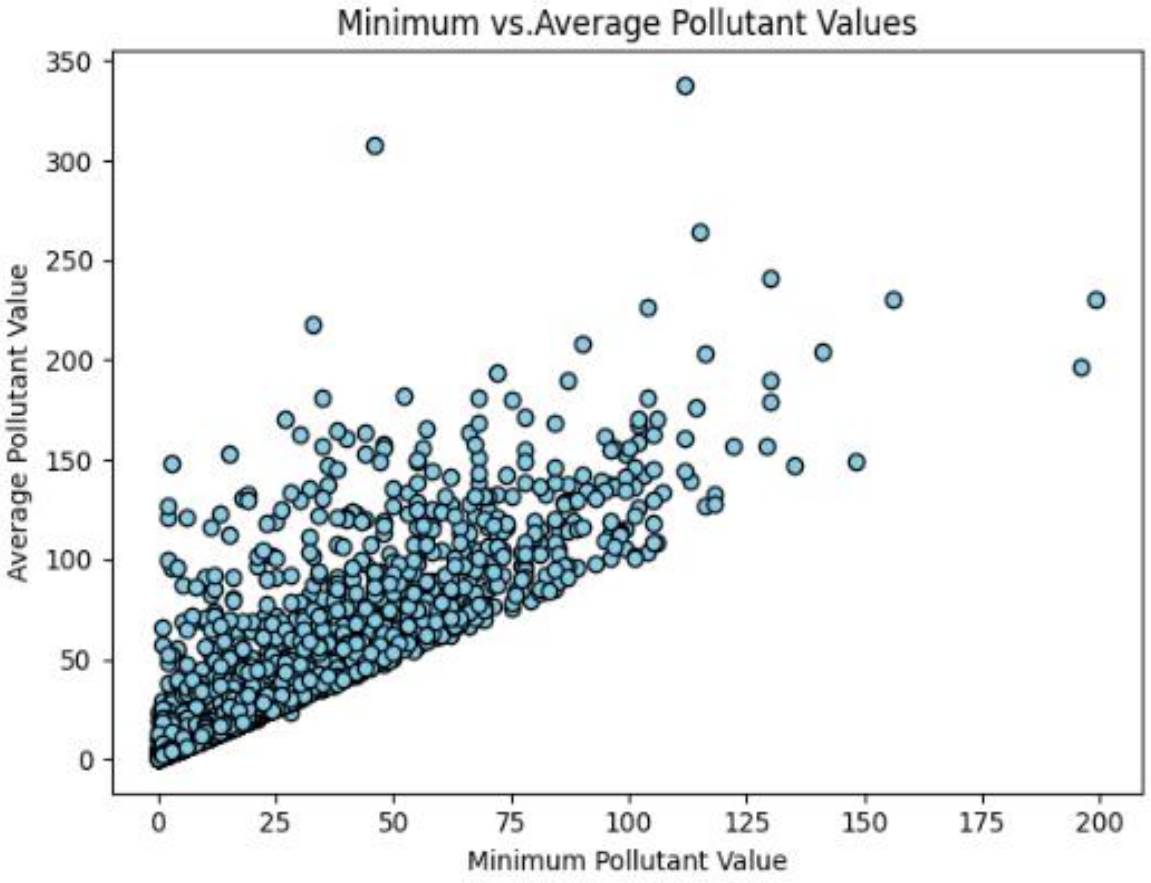
The box plot displays the median, spread, and potential outliers in average pollutant readings. Several observations lie above the upper whisker, indicating unusually high readings compared with the majority of the data. These observations may require further investigation.



4. Scatter Plot: Minimum vs. Average Pollutant Values

Library: Matplotlib | **Analysis:** Bivariate

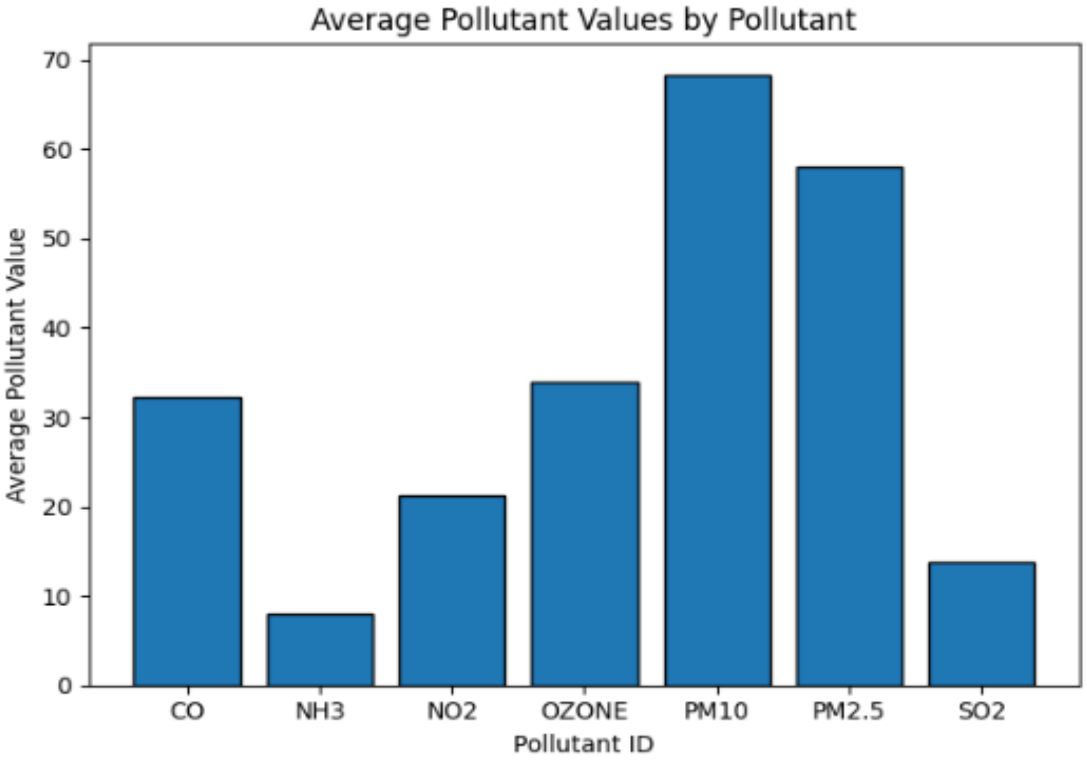
This scatter plot examines the relationship between minimum and average pollutant readings. The points show a positive relationship, meaning that higher minimum readings generally occur alongside higher average readings.



5. Bar Chart: Average Pollutant Values by Pollutant

Library: Matplotlib | **Analysis**: Bivariate

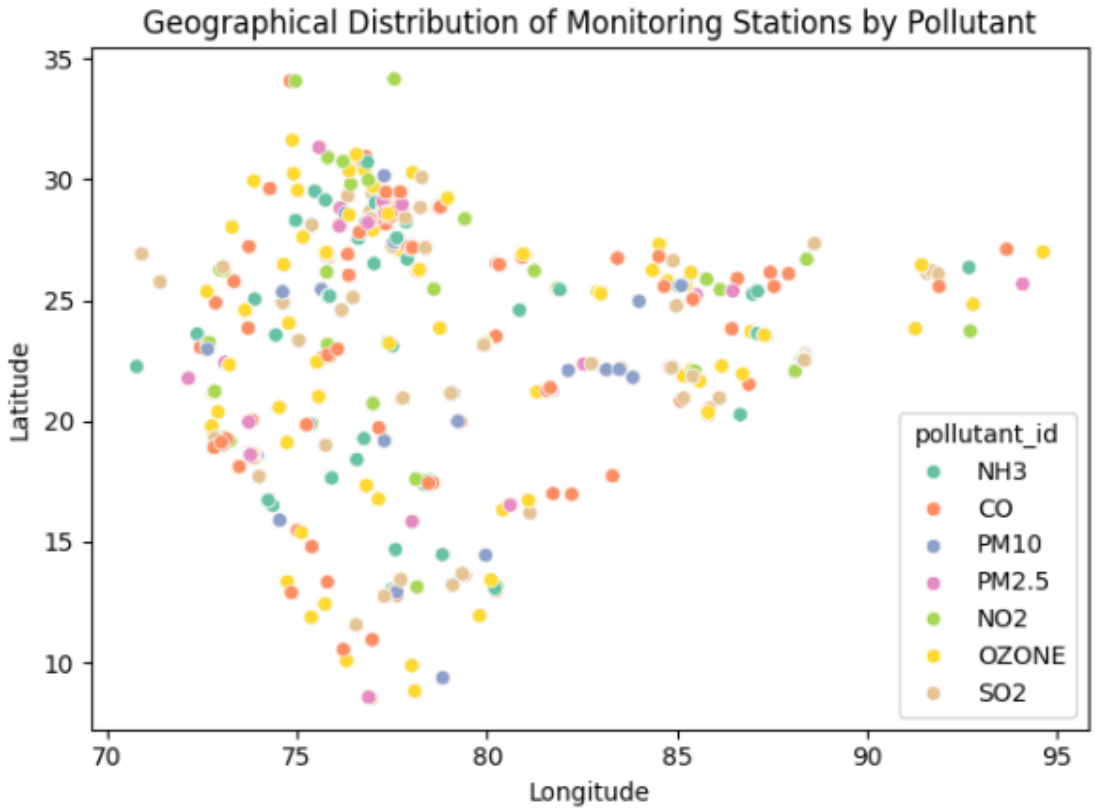
This bar chart compares the mean recorded value for each pollutant type. PM10 has the highest average value, followed by PM2.5, while NH3 has the lowest. These comparisons should be interpreted carefully because different pollutants may have different measurement scales.



6. Scatter Plot: Geographical Distribution of Monitoring Stations by Pollutant

Library: Seaborn | **Analysis:** Multivariate

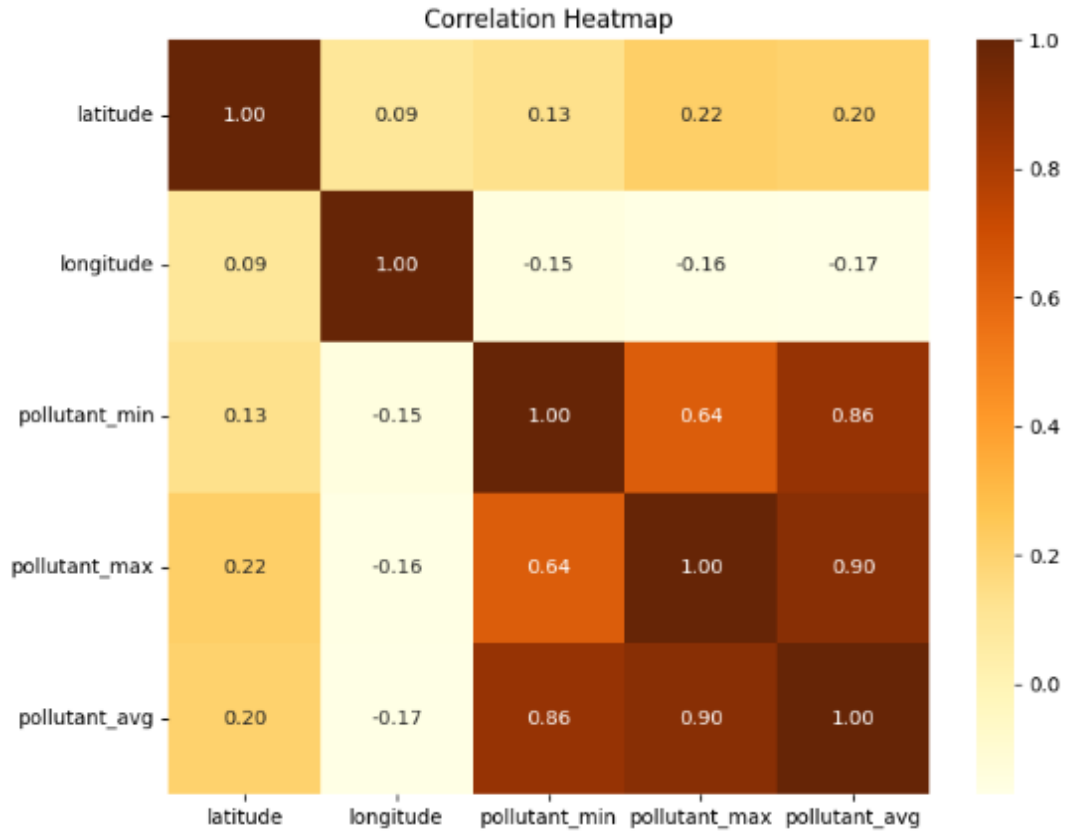
This scatter plot displays monitoring station locations using longitude and latitude, with colours representing pollutant types. The points are distributed across different parts of India, showing the geographical coverage of the monitoring stations and the pollutant categories recorded at those locations.



7. Heatmap: Correlation Between Numerical Variables

Library: Seaborn | **Analysis:** Multivariate

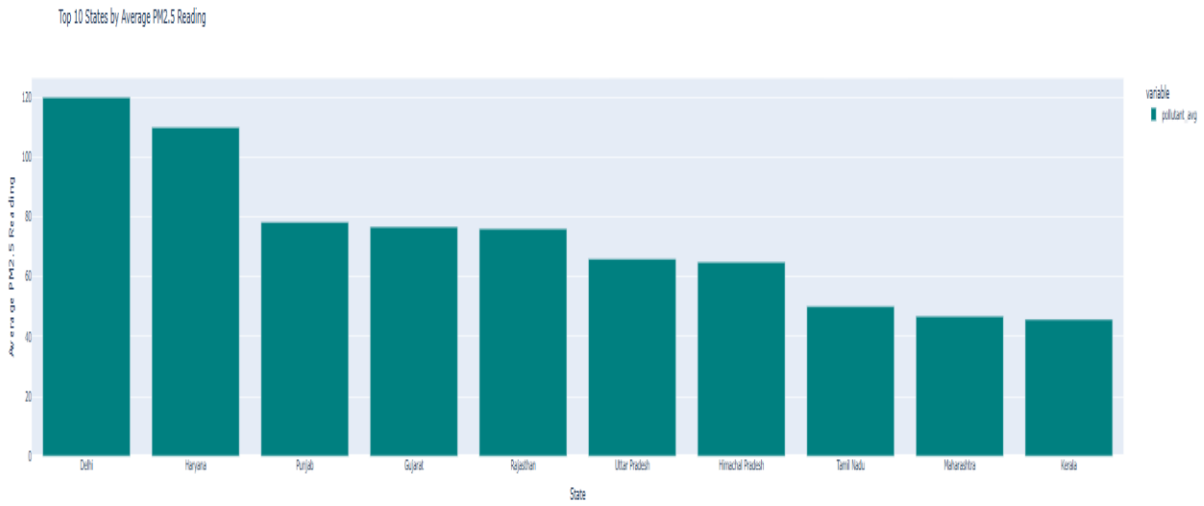
The correlation heatmap shows relationships between numerical variables. Average pollutant values have strong positive correlations with minimum values (0.86) and maximum values (0.90). Latitude has a weak positive correlation with average pollutant values (0.20), while longitude has a weak negative correlation (-0.17). Correlation indicates association, not causation.



8. Interactive Bar Chart: Average PM2.5 Values by State

Library: Plotly | **Analysis:** Bivariate

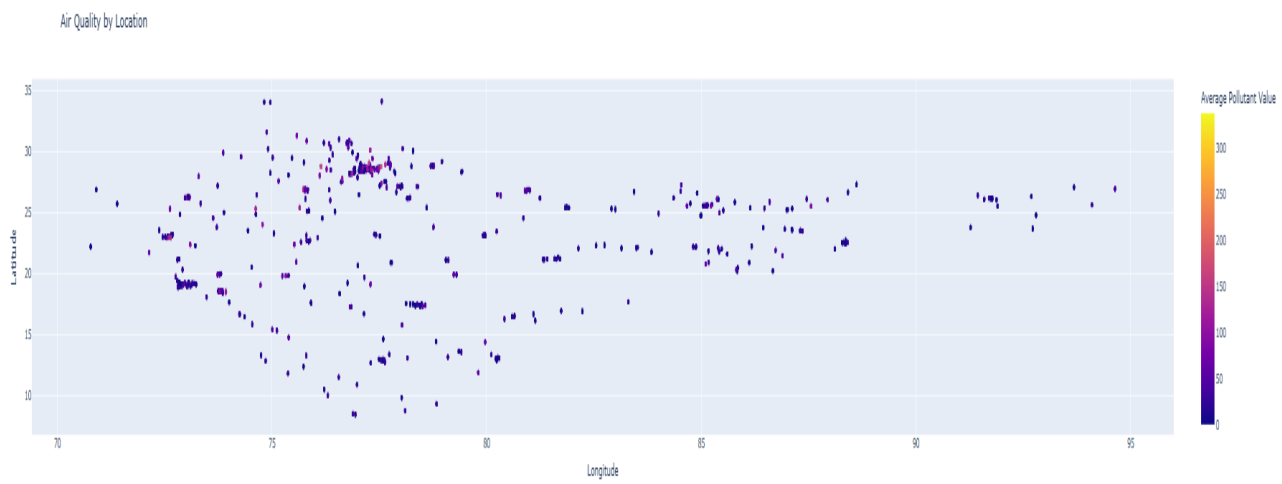
This interactive bar chart compares the average PM2.5 readings across states. Delhi has the highest average among the ten displayed states, followed by Haryana and Punjab. The chart allows users to explore differences in recorded PM2.5 levels between states.



9. Interactive Scatter Plot: Air Quality by Location

Library: Plotly | **Analysis:** Multivariate

This interactive scatter plot displays monitoring station locations using longitude and latitude. Colour represents the average pollutant reading, while hover details provide the station, city, and state. It helps users explore the geographical distribution of monitoring stations and differences in recorded pollutant values.



11. KEY FINDINGS AND INSIGHTS

The analysis of the air quality dataset produced the following findings:

- **Dataset Overview:** The original dataset contained 3,563 records. After data cleaning, 3,556 records remained for analysis, as seven records were removed during the cleaning process.
- **State-Wise Records:** Maharashtra had the highest number of records (602), followed by Uttar Pradesh (468) and Rajasthan (336).
- **City-Wise Records:** Delhi had the highest number of records (307), followed by Mumbai (154).
- **Pollutant-Wise Analysis:** PM10 had the highest average recorded value (68.35), followed by PM2.5 (58.05). These values should be interpreted carefully because different pollutants may have different measurement units or scales.
- **Average PM2.5 by State:** Delhi had the highest average PM2.5 reading (120.16), followed by Haryana (110.13) and Punjab (78.38) among the states shown in the analysis.

- **Percentage Analysis:** India accounted for 100% of records. Maharashtra had the highest state share (16.93%), while Delhi had the highest city share (8.63%). The dataset contained 508 monitoring stations.
- **Correlation Analysis:** A strong positive correlation was observed between minimum and average pollutant readings (0.86) and between maximum and average pollutant readings (0.90).
- **Geographical Analysis:** The interactive geographical plots helped explore the distribution of monitoring stations and variations in recorded pollutant values across locations. Overall, the findings show variations in pollutant readings and monitoring coverage across locations.

12. RECOMMENDATIONS

Based on the findings of the air quality analysis, the following recommendations are suggested:

- **Focus on PM10 and PM2.5:** Give special attention to monitoring these pollutants because they showed higher average values in the analysis.
- **Monitor High-Pollution Locations:** Locations showing higher pollutant values, including Delhi in the state-wise PM2.5 comparison, should be monitored regularly.
- **Improve Data Collection:** Collect accurate and updated air quality data to support more reliable analysis.
- **Analyse Changes Over Time:** Study pollutant levels across different dates and seasons to identify trends and variations.
- **Use Geographical Visualisation:** Use latitude and longitude to identify monitoring station locations and explore geographical patterns in pollutant values.
- **Support Further Investigation:** Use the findings to identify areas that may need closer monitoring and further investigation into possible pollution sources.

13. CONCLUSION

The Air Quality Data Analysis project was carried out to understand pollutant measurements and their variations across different locations using Python. The dataset was cleaned and analysed using Pandas and NumPy, while Matplotlib, Seaborn, and Plotly were used for data visualization.

The analysis showed that Maharashtra had the highest number of records, followed by Uttar Pradesh and Rajasthan. PM10 had the highest average recorded value of 68.35, followed by PM2.5 at 58.05. The state-wise analysis also showed that Delhi had the highest average PM2.5 reading among the states included in the analysis.

Strong positive correlations were observed between minimum and average pollutant readings and between maximum and average pollutant readings.

These findings show that pollutant measurements and monitoring coverage vary across locations. However, the results describe only the available dataset and do not necessarily represent current air quality conditions. Overall, the project provided practical experience in data cleaning, exploratory data analysis, statistical analysis, and data visualization. It also identified areas where additional data and further analysis could improve the understanding of air pollution patterns.

14. LIMITATIONS

- **Data Availability:** The analysis was limited to the records available in the dataset.
- **Missing and Inconsistent Data:** Missing values and inconsistencies may affect the accuracy of some results.
- **Limited Time Coverage:** The analysis may not represent current air quality conditions because continuous live data updates were not verified.
- **Geographical Coverage:** The findings depend on the locations and monitoring stations included in the dataset.
- **Correlation Limitations:** Correlation analysis identifies relationships between variables but does not establish causation.
- **Limited Information on Pollution Causes:** The dataset mainly contains pollutant measurements and location details. It does not provide sufficient information to confirm whether traffic, industrial activities, weather conditions, or other factors caused higher pollutant readings.

15. FUTURE SCOPE

- **Live Data Integration:** Connect the project to regularly updated air quality data sources to analyse more recent pollutant readings.
- **Predictive Analysis:** Apply machine learning techniques to predict future pollutant levels using suitable historical data.
- **Advanced Visualization:** Develop interactive dashboards with additional filters and geographical views to make the analysis easier to understand.
- **Time Based Analysis**: Analyse changes in pollutant levels across days, months, and seasons when suitable timestamp data is available.
- **Expanded Coverage:** Include more monitoring stations and locations to improve geographical comparisons.
- **Analysis of Pollution Causes:** In the future, additional data on traffic, industrial activities, weather conditions, and population density could be combined with air quality measurements. This would help investigate factors associated with higher pollutant readings in different locations.

16. REFERENCES

Dataset Source: https://www.data.gov.in/resource/real-time-air-quality-indexvarious-locations

Google Colab: Used for Python-based data cleaning, analysis, and visualization.

Seaborn: Used to create statistical data and informative visualizations. https://seaborn.pydata.org/